

HRMS Technical Documentation

Foundation, Employee Profile & Identity Administration

Professional implementation reference for developers, QA engineers, technical leads and system administrators

Documentation baseline: Identity Administration + Foundation master data + Employee profile integrations

Prepared from the implementation details established during the current HRMS development work. Code structure and terminology follow the current modular-monolith solution conventions.

1. Document Purpose

This document captures the implemented technical design and operational behavior for the Foundation master-data layer, the Employee profile domain, and the Identity administration/security slice. It is intended to let a new developer understand what exists, where it lives, how it behaves, how to call it, and where to make controlled changes.

1.1 Scope

Area	Status	Primary role
Foundation master data	Completed	Reusable reference/master data used by other modules
Employee profile	Completed	Core employee record and related profile information
Identity administration	Completed	Users, roles, permissions, assignment management
Permission authorization	Completed	Policy-based authorization using database permissions
Password management	Completed for change/reset	Secure password hashing and administrative reset

2. Solution Architecture

The current solution is a modular monolith. The application is split into API, shared BuildingBlocks, Persistence, and business modules. Application code uses repository abstractions and MediatR-style commands/queries; Infrastructure contains implementation details such as JWT, password hashing and authorization policy handling.

```
src/  
  HRMS.API  
  HRMS.BuildingBlocks  
  HRMS.Persistence  
Modules/  
  Identity  
    Application  
    Domain  
    Infrastructure  
  Foundation  
    Application  
    Domain  
    Infrastructure  
  Employee  
    Application  
    Domain  
    Infrastructure  
  Companies  
  Department  
  Attendance  
  Leave
```

2.1 Layer Responsibilities

Layer	Responsibility	Key conventions
API	HTTP endpoints, authentication middleware, controller-level	Controllers delegate to MediatR; policies

	authorization	via [Authorize(Policy=...)]
Application	Use cases, validation, business rules, DTOs	Commands/Queries + handlers + FluentValidation + BusinessRules
Domain	Entities and core domain model	Entity relationships and domain structure
Infrastructure	Security and EF configurations	JWT, password hashing, authorization handler, IEntityTypeConfiguration
Persistence	EF Core DbContext and generic repository implementations	IReadRepository/IWriteRepository -> Base repositories -> ApplicationDbContext
BuildingBlocks	Cross-cutting abstractions	Persistence contracts, audit base classes, pagination, behaviors

2.2 Request Processing Pattern

HTTP request

- > Controller
- > ASP.NET Core authorization (when policy is present)
- > MediatR command/query
- > FluentValidation pipeline
- > Business Rules
- > I/O through repository abstractions
- > Persistence / database
- > DTO / HTTP response

3. Foundation Module - Master Data

Foundation provides reference data that is shared by domain modules. These records are not business workflows; they exist to standardize values and enforce consistent foreign-key relationships.

Master	Primary consumers / purpose	Typical business rule
Country	Employee address	Unique, active reference value
State	Employee address	Belongs to a country/reference hierarchy
AddressType	EmployeeAddress	Classifies address records
Relationship	Emergency contacts, dependents, nominees	Standard relationship vocabulary
Gender	Employee personal information	Reference/master value
MaritalStatus	Employee personal information	Reference/master value
IdentifierType	Government identifiers	Defines identifier category
DocumentType	Employee documents	Defines document category
Skill	Employee skills	Reusable capability vocabulary
Certification	Employee certifications	Reusable certification master
Language	Employee languages	Reusable language master
EmploymentType	Employee	Classifies employment arrangement
EmploymentStatus	Employee	Tracks employment state

3.1 Foundation Integration Map

Country + State + AddressType
-> EmployeeAddress

Relationship
 -> EmergencyContact / Dependents / Nominees
 Gender + MaritalStatus
 -> EmployeePersonalInformation
 IdentifierType
 -> EmployeeGovernmentIdentifier
 DocumentType
 -> EmployeeDocument
 Skill -> EmployeeSkill
 Certification -> EmployeeCertification
 Language -> EmployeeLanguage
 EmploymentType + EmploymentStatus -> Employee

3.2 Expected Master-Data API Pattern

For each master entity that is exposed through the API, follow the project's established CRUD pattern: GET collection, GET by id, POST create, PUT update, and a business-safe deactivate operation where the master is not physically deleted.

Operation	HTTP	Expected behavior
List	GET /api/{Master}	Returns active/non-deleted master records according to the module query implementation
Get by id	GET /api/{Master}/{id}	Returns record or 404
Create	POST /api/{Master}	Validates required fields and uniqueness rules, then persists
Update	PUT /api/{Master}/{id}	Validates route/body id consistency and uniqueness exclusions
Deactivate	POST /api/{Master}/{id}/deactivate	Sets IsActive=false without physical deletion where supported

Exact controller names and payloads should be taken from the specific module implementation before extending an endpoint; do not invent a second endpoint naming convention.

4. Employee Module - Profile Domain

The Employee module is the central HR profile domain. It owns the employee record and the related profile collections that depend on Foundation reference data.

Employee area	Purpose	Foundation dependency
Employee	Core employee identity and employment data	EmploymentType, EmploymentStatus
Personal Information	Personal profile attributes	Gender, MaritalStatus
Address	Employee addresses	Country, State, AddressType
Contact	Phone/email contact data	Usually independent reference data
Emergency Contact	Emergency contact relationships	Relationship
Bank Account	Payroll/banking information	Usually independent reference data
Government Identifier	Official identifiers	IdentifierType
Dependent	Employee dependents	Relationship
Nominee	Nomination records	Relationship
Document	Employee document records	DocumentType
Education	Academic background	Employee-owned child records

Experience	Employment history	Employee-owned child records
Skill	Employee capabilities	Skill
Certification	Employee credentials	Certification
Language	Employee language records	Language

4.1 Typical Employee API Pattern

```
GET /api/Employee
GET /api/Employee/{id}
POST /api/Employee
PUT /api/Employee/{id}
DELETE /api/Employee/{id}
```

The existing Employee controller applies permission policies to the CRUD operations.

```
[Authorize(Policy = "Employee.View")]
[Authorize(Policy = "Employee.Create")]
[Authorize(Policy = "Employee.Update")]
[Authorize(Policy = "Employee.Delete")]
```

4.2 Employee Update Rules

- The route id must match the command id.
- Validation is handled before the handler performs persistence.
- Domain/business rules perform entity existence and related-reference checks where required.
- Writes flow through IWriteRepository rather than direct DbContext access from Application handlers.

5. Identity Module - Administration

Identity now covers user accounts, roles, permissions, assignment management, and password operations.

Authentication remains JWT-based. Authorization is permission-based and policy-driven.

5.1 User Administration

Operation	HTTP	Payload / behavior
Create	POST /api/User	employeeId, userName, password; creates account with hashed password
List	GET /api/User	Returns UserDto without PasswordHash
Get	GET /api/User/{id}	Returns UserDto without PasswordHash
Update	PUT /api/User/{id}	id, userName, isActive; route id must match body id
Deactivate	POST /api/User/{id}/deactivate	Sets IsActive=false
Change password	POST /api/User/{id}/change-password	Requires currentPassword + newPassword
Reset password	POST /api/User/{id}/reset-password	Requires permission User.ResetPassword; only newPassword is required

5.2 Create User Example

```
{
  "employeeId": "{employee-guid}",
  "userName": "anzar.admin",
```

```
"password": "Password@123"
}
```

Expected success: HTTP 201 Created with the new user id. Password is converted to PasswordHash using the existing IPasswordHasher implementation.

5.3 Change Password Example

```
POST /api/User/{userId}/change-password
{
  "userId": "{user-guid}",
  "currentPassword": "CurrentPassword@123",
  "newPassword": "NewPassword@456"
}
```

Expected success: HTTP 204 No Content. A wrong current password produces an unauthorized error rather than silently resetting the account.

5.4 Reset Password Example

```
POST /api/User/{userId}/reset-password
Authorization: Bearer {admin-jwt}
{
  "userId": "{user-guid}",
  "newPassword": "Password@123"
}
```

Expected success: HTTP 204 No Content. This operation intentionally does not require the old password, but it is protected by the database-backed User.ResetPassword permission.

6. Roles and Permissions

6.1 Role Administration

Operation	HTTP	Example payload
Create	POST /api/Role	{ "name": "Manager", "description": "..." }
List	GET /api/Role	Returns RoleDto records
Get	GET /api/Role/{id}	Returns RoleDto
Update	PUT /api/Role/{id}	{ "id": "{role-guid}", "name": "Manager", "description": "...", "isActive": true }
Deactivate	POST /api/Role/{id}/deactivate	No body

6.2 Permission Administration

Operation	HTTP	Example payload
Create	POST /api/Permission	{ "name": "User.ResetPassword", "description": "Reset a user password" }
List	GET /api/Permission	Returns PermissionDto records
Get	GET /api/Permission/{id}	Returns PermissionDto
Update	PUT /api/Permission/{id}	{ "id": "{permission-guid}", "name": "...", "description": "...", "isActive": true }
Deactivate	POST /api/Permission/{id}/deactivate	No body

6.3 User -> Role

Operation	HTTP	Payload
Assign	POST /api/UserRoles	{ "userId": "{user-guid}", "roleId": "{role-guid}" }
Get user roles	GET /api/UserRoles/user/{userId}	No body
Remove	DELETE /api/UserRoles/{userId}/{roleId}	No body

6.4 Role -> Permission

Operation	HTTP	Payload
Assign	POST /api/RolePermissions	{ "roleId": "{role-guid}", "permissionId": "{permission-guid}" }
Get role permissions	GET /api/RolePermissions/role/{roleId}	No body
Remove	DELETE /api/RolePermissions/{roleId}/{permissionId}	No body

7. Permission-Based Authorization

The original permission authorization design was recovered from Git history and restored into the current modular Identity Infrastructure. Policies are defined using `PermissionRequirement` and evaluated by `PermissionAuthorizationHandler`.

7.1 Authorization Flow

```
User
-> JWT Authentication
-> Current user context (userId / employeeId / userName)
-> User.UserRoles
-> active Role.RolePermissions
-> active Permission records
-> Permission.Name == required policy name
-> context.Succeed(requirement)
```

7.2 Example

```
[Authorize(Policy = "Employee.Update")]
```

This policy is satisfied only when the authenticated user has a role whose active `RolePermissions` include an active `Permission` named `Employee.Update`.

7.3 Security Test Matrix

Scenario	Expected result
No JWT token	401 Unauthorized
Valid JWT + required permission	200/2xx as appropriate
Valid JWT + missing permission	403 Forbidden
Inactive user	Login blocked / unauthorized
User with no role	Login blocked by current <code>IdentityBusinessRules</code>
Inactive role or permission	Permission not granted

8. Password Security

The existing PasswordHasher uses PBKDF2 with SHA-256, a 16-byte random salt, a 32-byte derived hash, and 600,000 iterations. The stored value format is iterations.salt.hash. Existing IPasswordHasher abstraction exposes Hash and Verify.

```
IPasswordHasher
    string Hash(string password)
    bool Verify(string password, string passwordHash)
```

Do not expose PasswordHash in API DTOs. Password reset and change operations should only work through commands/handlers and the existing hashing abstraction.

9. Persistence and Auditing

Normal auditable entities use the established repository pattern: IReadRepository / IWriteRepository backed by BaseReadRepository / BaseWriteRepository and ApplicationDbContext.

```
Application handler
    -> IReadRepository / IWriteRepository
    -> BaseReadRepository / BaseWriteRepository
    -> ApplicationDbContext
```

UserRole and RolePermission were converted to AuditableEntity<Guid> so they conform to the existing repository contract and to the application-wide audit/soft-delete behavior. Their relationship uniqueness remains enforced by filtered unique indexes on the active relationship pairs.

9.1 Audit Fields

Field	Purpose
CreatedBy / CreatedOn	Who and when created the record
ModifiedBy / ModifiedOn	Who and when last modified the record
DeletedBy / DeletedOn	Who and when performed soft deletion
IsDeleted	Logical deletion flag

10. Database / EF Migration Baseline

The current database has an established EF migration history through the Identity assignment/audit changes. Migration work must follow the existing practice: generate migration, inspect generated SQL/operations, preserve existing data, apply update, then confirm model synchronization.

10.1 Safe Migration Checklist

- Run dotnet build before migration generation.
- Run dotnet ef migrations has-pending-model-changes before and after changes.
- Inspect generated migration when altering existing populated tables.
- Preserve existing relationship data; current baseline contained 3 UserRoles and 11 RolePermissions rows at the time of the assignment audit conversion.
- Run dotnet ef database update only after validating the migration operations.

11. Developer Change Guide

Use the following decision rules when extending the implemented areas.

Need	Where to change	Do not do
New use case	Application/Features/<area>/Commands or Queries	Do not place business logic in controllers
Business invariant	Existing BusinessRules class for the domain/application feature	Do not duplicate rule logic across controllers
Validation	CommandValidator / QueryValidator	Do not bypass the validation pipeline
Database mapping	Infrastructure/Configurations	Do not hard-code schema rules in controllers
Persistence	Existing IReadRepository/IWriteRepository pattern	Do not access ApplicationDbContext from Application handlers
Permission-protected API	Controller [Authorize(Policy = "...")]	Do not replace with unrelated role checks without architectural review
New master data	Foundation module when genuinely reusable	Do not duplicate master values inside Employee

12. Recommended Improvements / Next Quality Gates

These are architecture and documentation improvements, not claims that the corresponding feature is already implemented.

- Standardize permission names by module and operation, e.g. Employee.View / Employee.Create / Employee.Update / Employee.Delete and Identity administration equivalents.
- Add automated integration tests for 401, 403, and successful permission paths.
- Add API documentation for each Identity administration endpoint with sample requests/responses and authorization requirements.
- Consider a formal password-forgotten flow with time-limited reset tokens and a trusted out-of-band channel when email/notification infrastructure exists; do not equate it with Change Password.
- Add a release checklist covering migrations, authorization regression tests, and audit/soft-delete behavior.

13. End-to-End Functional Examples

13.1 Employee View

GET /api/Employee

Authorization: Bearer {employee-jwt}

Expected: 200 OK when the user is authenticated and has Employee.View through an active role/permission chain.

13.2 Employee Update

PUT /api/Employee/{employeeId}

Authorization: Bearer {employee-jwt}

{ existing UpdateEmployeeCommand payload }

Expected: authorization succeeds only for a user with Employee.Update; the command then passes validation and business rules before persistence.

13.3 Admin Reset Password

```
POST /api/User/{userId}/reset-password
Authorization: Bearer {admin-jwt}
{
  "userId": "{user-guid}",
  "newPassword": "Password@123"
}
```

Expected: 204 No Content when the caller has User.ResetPassword and the target user exists.

14. Implementation Status Snapshot

Capability	Status
Foundation master data: 13 entities listed in scope	Completed
Employee profile and related records listed in scope	Completed
User CRUD and account administration	Completed
Change Password	Completed
Reset Password	Completed
Role CRUD	Completed
Permission CRUD	Completed
User -> Role assignment + views	Completed
Role -> Permission assignment + views	Completed
Permission-based authorization	Completed
401/authorized 200 verification	Verified during development
403 negative authorization verification	Should remain a regression test
Technical cleanup / release baseline	Ready for final Git checkpoint

15. Developer Quick Reference

Core patterns

Controller -> MediatR -> Validation -> Business Rules -> Repository -> DB

Authorization

[Authorize(Policy = "Module.Operation")]

Repositories

IReadRepository<TEntity,TKey>

IWriteRepository<TEntity,TKey>

Audit base

AuditableEntity<TKey> : BaseEntity<TKey>

Identity relationship model

User -> UserRoles -> Role -> RolePermissions -> Permission

Password

IPasswordHasher.Hash / Verify

End of technical baseline.